EdgeSentinel

Resilient Edge-Cloud Anomaly Detection Platform

Part 1 — Production Foundation

---

1. Project Overview

EdgeSentinel is an edge-cloud platform designed to monitor simulated industrial machines, detect abnormal behavior, and intelligently process telemetry at the edge or in the cloud.

The project is intentionally designed as a **production-oriented student project**.

The objective is not to build a huge enterprise system with unnecessary technologies.

The objective is to demonstrate that a student understands how to take a working prototype and progressively improve it into a system that could realistically move toward deployment.

Core idea

Industrial devices continuously generate telemetry such as:

- Temperature
- Humidity
- Vibration
- Pressure
- Machine state
- Timestamp
- Device identity

The telemetry is sent through MQTT to an edge service.

The edge service performs local anomaly detection and decides how the event should be processed.

Possible decisions include:

EDGE_ONLY

CLOUD_ONLY

HYBRID

The system should continue operating when cloud connectivity is temporarily unavailable.

When connectivity returns, locally stored events should synchronize with the cloud.

---

2. Why This Project Exists

The project is intended to demonstrate practical knowledge in:

- Software architecture
- Backend development
- Distributed systems
- Edge computing
- Cloud concepts
- AI/ML integration
- API design
- Authentication and authorization
- Database design
- Docker
- Testing
- Observability
- DevOps
- Reliability engineering

However, **technology count is not the goal**.

Every technology introduced into the system must solve a real problem.

For example:

PostgreSQL is used because telemetry and system state need durable persistence.

MQTT is used because the system represents device-to-edge communication.

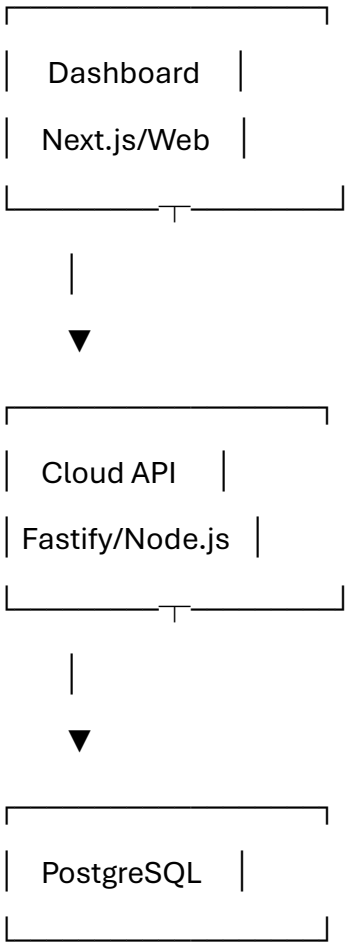
SQLite is used at the edge because the edge node must continue operating during temporary cloud/network failures.

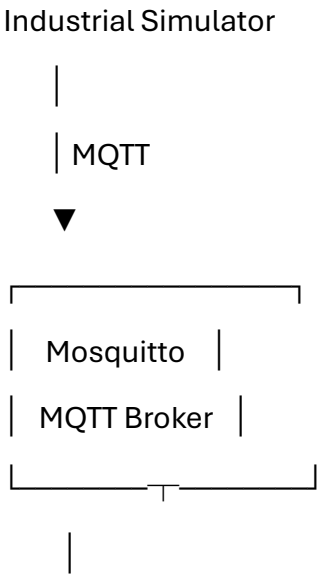
Prometheus is used because system behavior needs to be measurable.

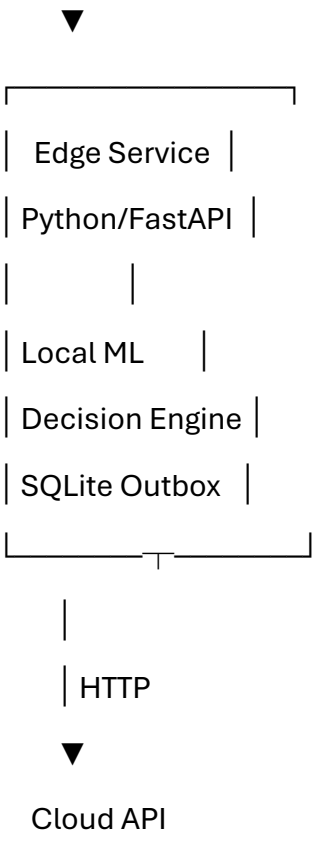
---

3. Current System

The initial prototype already contains the following conceptual components:







The existing prototype is **not yet considered production-ready**.

Part 1 will establish the foundation required before moving into advanced reliability and DevOps work.

---

4. Part 1 Objective

The objective of Part 1 is:

Convert the existing prototype into a clean, maintainable, persistent, secure and testable application foundation.

At the end of Part 1:

- telemetry must be persisted properly
- API configuration must be externalized
- secrets must not be hardcoded
- authentication must be improved
- authorization must exist
- API validation must be consistent
- database migrations must exist

- services must have proper health checks
- important business logic must have tests
- Docker services must be reproducible
- the project structure must be understandable to another developer

Part 1 does **not** attempt to implement the complete production platform.

---

5. Scope of Part 1

Included

Application foundation

- Clean project structure
- Environment-based configuration
- PostgreSQL persistence
- Database migrations
- Proper API validation
- Authentication
- Basic role-based authorization
- Error handling
- Health checks
- Structured logging foundation
- Unit tests
- Integration tests
- Docker improvements
- Documentation

Edge foundation

- Clean edge-service structure
- Persistent local outbox
- Reliable event identifiers
- Retry-safe synchronization

- Configuration through environment variables
- Graceful shutdown
- Basic health endpoint

Development quality

- Linting
- Formatting
- Automated tests
- Type checking
- Docker Compose validation
- CI foundation

---

6. Explicitly NOT in Part 1

Do **not** introduce these yet:

- Kubernetes
- Terraform
- Helm
- Argo CD
- Kafka
- Redis
- Service mesh
- Multi-cloud
- Blockchain
- Complex microservice decomposition
- Complex event streaming architecture
- ML model registry
- Advanced MLOps
- Multi-region deployment
- Kubernetes operators

- Enterprise IAM platforms

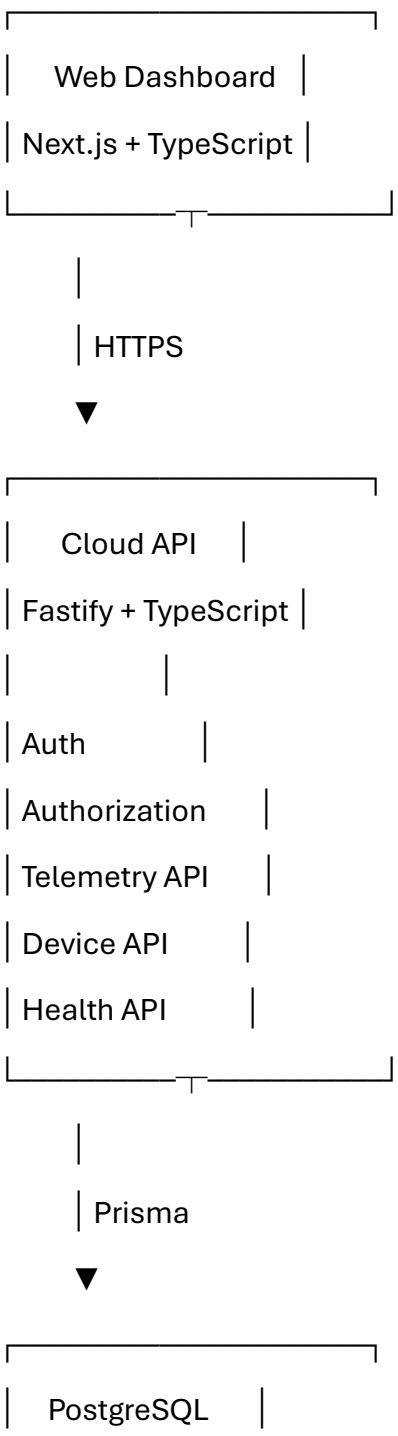
These may be considered later **only if there is a real requirement**.

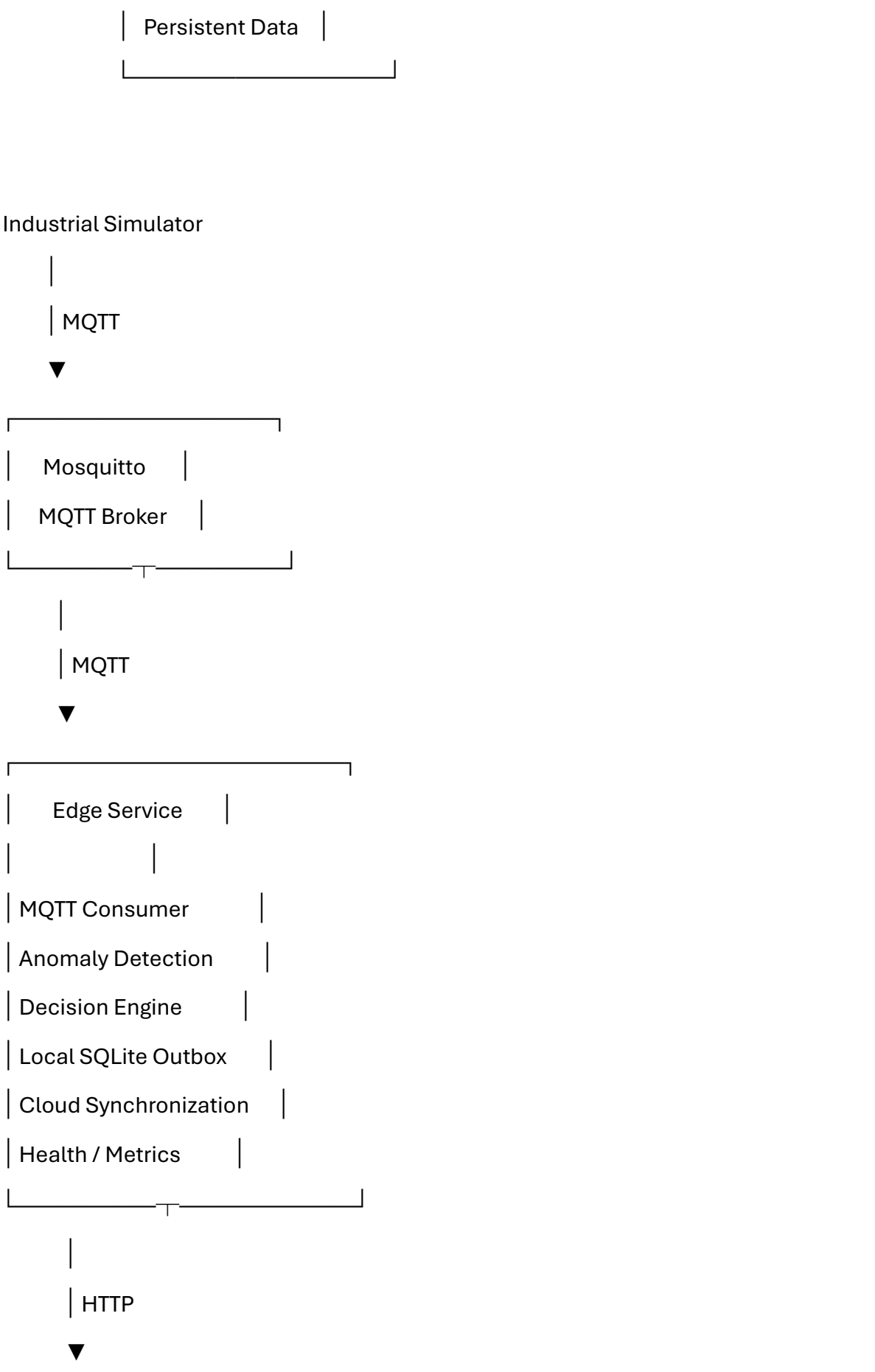
The project should remain understandable to a talented student.

---

7. Target Architecture for Part 1

The architecture should remain relatively simple.




Persistent Data
Industrial Simulator
MQTT
Mosquitto
MQTT Broker
MQTT
Edge Service
MQTT Consumer
Anomaly Detection
Decision Engine
Local SQLite Outbox
Cloud Synchronization
Health / Metrics
HTTP

Cloud API

---

8. Architectural Principles

Part 1 should follow these principles.

8.1 Keep services separated by responsibility

The project should have clear boundaries:

Web

↓

API

↓

Database

Simulator

↓

MQTT

↓

Edge

↓

API

Do not mix:

- database logic with HTTP controllers
- ML logic with MQTT connection code
- authentication logic with business logic
- configuration values directly inside application code

---

9. Backend Architecture

The Cloud API should use a simple layered structure.

Example:

```
apps/api/

src/
├── config/
│   ├── env.ts
│   └── logger.ts
│
├── plugins/
│   ├── auth.ts
│   └── database.ts
│
├── modules/
│   ├── auth/
│   │   ├── auth.controller.ts
│   │   ├── auth.service.ts
│   │   ├── auth.schema.ts
│   │   └── auth.types.ts
│   │
│   ├── telemetry/
│   │   ├── telemetry.controller.ts
│   │   ├── telemetry.service.ts
│   │   ├── telemetry.repository.ts
│   │   ├── telemetry.schema.ts
│   │   └── telemetry.types.ts
│   │
│   └── devices/
│       ├── device.controller.ts
```

```
│      ├── device.service.ts
│      ├── device.repository.ts
│      └── device.schema.ts
│
├── common/
│   ├── errors/
│   ├── middleware/
│   └── utils/
│
├── routes/
│
└── server.ts
```

The exact structure can differ slightly.

The important thing is the separation of responsibilities.

---

10. Database Foundation

The prototype must stop relying on temporary in-memory telemetry storage.

PostgreSQL becomes the primary persistent database.

Minimum entities

Part 1 should introduce only the entities actually required.

Users

users

---

id

email

password_hash

role

created_at

updated_at

Roles:

ADMIN

OPERATOR

VIEWER

Devices

devices

----

id

device_id

name

status

created_at

updated_at

Telemetry

telemetry

----

id

event_id

device_id

temperature

humidity

vibration

pressure

machine_state

timestamp

created_at

Anomaly Events

anomaly_events

---

id

telemetry_id

severity

score

decision

model_version

created_at

The schema should include appropriate:

- primary keys
- foreign keys
- unique constraints
- indexes
- timestamps

---

11. Database Requirements

Use:

- PostgreSQL
- Prisma
- migrations

The application must be able to start with a clean database.

Example workflow:

Clone repository

↓

Install dependencies

↓

Start Docker Compose

↓

Run database migrations

↓

Start application

↓

Application works

A developer should not need to manually create tables.

---

12. Idempotency

Telemetry events must contain a unique eventId.

Example:

```
{
  "eventId": "evt-001",
  "deviceId": "machine-001",
  "temperature": 74.2,
  "humidity": 62.1,
  "vibration": 0.82,
  "pressure": 101.2,
  "machineState": "RUNNING",
  "timestamp": "2026-09-05T10:30:00Z"
}
```

If the same event is submitted twice:

Request 1 → stored

Request 2 → recognized as duplicate

The system must not create duplicate telemetry records.

This is particularly important because the edge service may retry events.

---

13. Configuration Management

No application secrets should be hardcoded.

Bad:

admin:password

JWT_SECRET = "my-secret"

DATABASE_URL = "..."

Instead:

.env

Example:

NODE_ENV=development

PORT=3000

DATABASE_URL=...

JWT_SECRET=...

MQTT_BROKER_URL=...

EDGE_API_URL=...

Applications should validate required environment variables during startup.

If required configuration is missing:

Application startup

↓

Configuration validation

↓

Missing required variable

↓

Clear startup error

↓

Application exits

Do not silently use unsafe defaults for secrets.

---

14. Authentication

The API must require authentication for protected operations.

Minimum flow:

User

│

│ Login

▼

POST /api/v1/auth/login

│

▼

Authentication

│

▼

Access token/session

│

▼

Protected API

Authentication should no longer depend on a hardcoded production credential.

For Part 1, a simple database-backed user authentication system is sufficient.

Password requirements:

- never store plaintext passwords

- hash passwords using an appropriate password hashing algorithm
- never return password hashes through API responses

---

15. Authorization

Authentication answers:

Who are you?

Authorization answers:

What are you allowed to do?

Part 1 introduces basic RBAC.

ADMIN

Can:

- manage users
- manage devices
- view telemetry
- view anomalies

OPERATOR

Can:

- view devices
- view telemetry
- view anomalies

VIEWER

Can:

- view telemetry
- view anomalies

The authorization logic should exist independently from individual route handlers where practical.

---

16. API Design

API endpoints should follow consistent conventions.

Example:

/api/v1/auth/login

/api/v1/devices

/api/v1/devices/:id

/api/v1/telemetry

/api/v1/telemetry/:id

/api/v1/anomalies

/api/v1/health

Use appropriate HTTP methods:

GET

POST

PATCH

DELETE

Use consistent response formats.

Example successful response:

```
{
  "data": {
    "id": "123",
    "deviceId": "machine-001"
  }
}
```

Example error:

```
{
  "error": {
    "code": "DEVICE_NOT_FOUND",
    "message": "Device was not found"
  }
}
```

Exact response format can be adjusted, but it must remain consistent across the API.

---

17. API Validation

Every externally supplied input must be validated.

Validate:

- request body
- query parameters
- path parameters
- authentication input

Example:

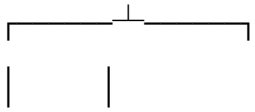
POST /telemetry

↓

Validate schema

↓

Valid?



YES NO

| |

▼ ▼

Process 400

Zod or an equivalent schema-validation library may be used.

Never trust data simply because it came from the edge service.

---

18. Error Handling

The API should have centralized error handling.

Expected categories:

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

409 Conflict

422 Validation Error

429 Rate Limited

500 Internal Server Error

Part 1 does not require sophisticated distributed error handling.

It does require predictable and understandable API behavior.

---

19. Health Checks

Each important service should expose a health endpoint.

Example:

GET /health

The API health check should verify basic application availability.

A readiness-style check may verify dependencies such as PostgreSQL.

Example:

{

"status": "ok",

"database": "connected"

}

The purpose is to allow Docker and future deployment platforms to determine whether a service is functioning.

---

20. Edge Service Structure

The edge service should also have clear responsibilities.

Suggested structure:

services/edge/

app/

├— config/

├— mqtt/

├— inference/

├— decision/

├— storage/

├— sync/

├— health/

├— logging/

└— main.py

Responsibilities:

MQTT

Receives telemetry.

Inference

Runs the anomaly detection model.

Decision Engine

Determines:

EDGE_ONLY

CLOUD_ONLY

HYBRID

Storage

Stores locally required information.

Sync

Sends pending events to the Cloud API.

Health

Reports service state.

---

21. Edge Offline-First Requirement

The edge service must continue processing telemetry when the cloud API is unavailable.

Example:

Device

↓

MQTT

↓

Edge

↓

Local inference

↓

Anomaly detected

↓

Cloud unavailable

↓

SQLite Outbox

↓

Cloud returns

↓

Retry

↓

Cloud API

The edge must not simply lose the event because the API is temporarily unavailable.

---

22. Local Outbox Requirements

The SQLite outbox should contain at least:

id

event_id

payload

status

attempt_count

created_at

updated_at

last_error

Possible states:

PENDING

PROCESSING

SENT

FAILED

Part 1 should implement the basic reliable flow:

Create event

↓

Store locally

↓

Attempt synchronization

↓

Success → mark sent

↓

Failure → retry later

The synchronization process must be safe against duplicate submission.

---

23. Retry Behavior

Retries must not happen in an uncontrolled loop.

Use a basic retry strategy such as:

Attempt 1

↓

wait

Attempt 2

↓

wait longer

Attempt 3

↓

...

Exponential backoff may be used.

The implementation should avoid:

```
while failure:
  retry immediately
```

because that can overload the API during an outage.

Advanced retry policies can be introduced later.

---

24. Graceful Shutdown

The edge service should handle shutdown signals.

Example:

SIGTERM

↓

Stop accepting new work

↓

Finish safe processing

↓

Close MQTT connection

↓

Close SQLite

↓

Flush logs

↓

Exit

The API should similarly close:

- HTTP server
- database connection
- other resources

This becomes important during Docker/cloud deployments.

---

25. Logging

Introduce structured logging.

Example:

```
{
  "level": "info",
  "service": "edge",
  "event": "telemetry_processed",
  "eventId": "evt-001",
```

"deviceId": "machine-001",

"decision": "EDGE_ONLY"

}

Avoid logs such as:

something went wrong

Logs should provide enough context to understand what happened.

Do not log:

- passwords
- JWT secrets
- sensitive credentials
- unnecessary tokens

---

26. Correlation / Event IDs

Every telemetry event should have a stable identifier.

Example:

eventId = evt-8f2a...

This identifier should allow us to follow an event through the system:

Simulator

↓

MQTT

↓

Edge

↓

Outbox

↓

Cloud API

↓

PostgreSQL

↓

Dashboard

This will become even more important when distributed tracing is introduced in Part 2.

---

27. Testing Strategy

Part 1 must establish a proper testing foundation.

Testing should exist at multiple levels.

Unit tests

Test isolated business logic.

Examples:

Decision Engine

Severity calculation

Validation logic

Retry calculation

Authorization rules

Integration tests

Test components working together.

Examples:

API + PostgreSQL

API authentication

Telemetry insertion

Duplicate event handling

Basic end-to-end test

Verify the main path:

Simulator

↓

MQTT

↓

Edge

↓

Cloud API

↓

PostgreSQL

The goal is not hundreds of tests.

The goal is to demonstrate that the **important behavior is tested**.

---

28. Docker Requirements

Docker Compose remains the primary local environment.

Expected services:

mosquitto

postgres

simulator

edge-service

cloud-api

web

prometheus

grafana

However, Part 1 focuses primarily on:

mosquitto

postgres

simulator

edge-service

cloud-api

web

Observability improvements will be expanded in Part 2.

---

29. Docker Improvements

Each service should have:

- appropriate Dockerfile
- predictable startup
- environment configuration
- health check where appropriate
- non-root execution where practical
- no hardcoded secrets

Avoid unnecessary complexity.

The objective is:

git clone

↓

docker compose up

↓

system starts

---

30. Development Workflow

Use a normal software development workflow.

Issue

↓

Branch

↓

Implementation

↓

Unit tests

↓

Integration tests

↓

Pull Request

↓

CI

↓

Review

↓

Merge

Example branches:

main

develop

feature/database-persistence

feature/rbac

feature/edge-outbox

Do not create complicated Git branching strategies unnecessarily.

---

31. CI Foundation

GitHub Actions should automatically perform basic validation.

On every pull request:

Checkout

↓

Install dependencies

↓

Lint

↓

Type check

↓

Unit tests

↓

Build

For the backend:

TypeScript

Lint

Tests

Build

For the edge:

Python

Lint

Tests

For the web:

TypeScript

Lint

Build

Advanced security scanning will be introduced in Part 2.

---

32. Documentation Requirements

The project should be understandable to another developer.

The repository should contain:

README.md

docs/

├— architecture.md

├── api.md

├── database.md

├── development.md

└── decisions/

Documentation should explain:

- what the project does
- why the architecture exists
- how to run it
- how the services communicate
- how the database is structured
- how authentication works
- how to run tests
- known limitations

---

33. Architecture Decision Records

Important architectural decisions should be documented.

Example:

docs/decisions/

Possible ADRs:

ADR-001-use-mqtt.md

ADR-002-use-postgresql.md

ADR-003-edge-local-outbox.md

ADR-004-use-nextjs-dashboard.md

ADR-005-use-fastify-api.md

Each ADR should answer:

Context

Decision

Why

Alternatives considered

Consequences

This is valuable for an internship portfolio because it demonstrates **engineering reasoning**, not just implementation.

---

34. Definition of Done — Part 1

Part 1 is complete only when all of the following are true.

Architecture

- Service responsibilities are clearly separated
- API structure is understandable
- Edge structure is understandable
- No unnecessary services have been introduced

Database

- PostgreSQL is the persistent application database
- Prisma schema exists
- Database migrations exist
- Telemetry is persisted
- Devices are persisted
- Users are persisted
- Anomaly events are persisted
- Appropriate indexes exist
- Foreign keys exist
- Duplicate telemetry is prevented

Configuration

- Secrets are removed from source code
- Environment variables are used
- Configuration is validated during startup
- Example environment file exists

- .env is excluded from Git

Authentication

- Database-backed users exist
- Passwords are hashed
- Login works
- Protected endpoints require authentication
- No hardcoded production credentials exist

Authorization

- ADMIN role exists
- OPERATOR role exists
- VIEWER role exists
- Role permissions are enforced

API

- API versioning exists
- Request validation exists
- Consistent error responses exist
- HTTP status codes are appropriate
- Health endpoint exists
- API documentation exists

Edge

- Edge service has clear internal modules
- SQLite outbox persists events
- Events have unique IDs
- Retry behavior exists
- Duplicate submissions are safe
- Graceful shutdown exists
- Edge configuration uses environment variables

Testing

- Unit tests exist
- Integration tests exist
- Critical decision logic is tested
- Authentication is tested
- Telemetry persistence is tested
- Duplicate event behavior is tested
- Main telemetry flow is tested

Docker

- Docker Compose starts successfully
- Services have health checks where appropriate
- Configuration is externalized
- Containers do not require manual database setup

CI

- Pull requests trigger CI
- Lint runs
- Tests run
- Build runs
- Failed checks prevent merging

Documentation

- README updated
- Architecture documented
- Database documented
- API documented
- Development setup documented
- Important architecture decisions documented

---

35. Part 1 Success Criteria

The most important test is:

Can another developer clone the repository and understand, run, test and modify the system without asking the original developer what everything means?

A second test:

If PostgreSQL is restarted, does the application recover without losing previously committed telemetry?

A third:

If the cloud API is unavailable, does the edge continue processing and later synchronize its pending events?

A fourth:

Can an unauthorized user access protected resources?

A fifth:

Can a duplicate telemetry event create duplicate records?

A sixth:

Can the complete system be validated automatically through CI?

If the answer to these questions is yes, Part 1 has achieved its purpose.

---

36. What We Should Be Able to Demonstrate After Part 1

At the end of Part 1, the project should be demonstrable as:

1. Start the system

2. Authenticate as an administrator

3. Register/view devices

4. Start the industrial simulator

5. Generate telemetry

6. Edge processes telemetry

7. Anomaly detection runs

8. Telemetry reaches the API

9. PostgreSQL persists the data

10. Dashboard displays the data

11. Stop the cloud API

12. Edge continues processing

13. Events accumulate in SQLite

14. Restart the API

15. Events synchronize

16. Duplicate events are safely handled

17. Tests pass

18. CI passes

This is a much stronger internship demonstration than simply showing a dashboard.

---

37. Part 1 Deliverables

When Part 1 is complete, the repository should contain:

```
EdgeSentinel/
│
├── apps/
│   ├── api/
│   └── web/
│
├── services/
│   ├── edge/
│   └── simulator/
│
├── infrastructure/
│   └── docker/
│
├── prisma/
│
├── docs/
│   ├── architecture.md
│   ├── api.md
│   ├── database.md
│   ├── development.md
│   └── decisions/
│
├── .github/
│   └── workflows/
│
├── docker-compose.yml
├── .env.example
```

├── .gitignore

└── README.md

The exact folder structure may be adapted to the current repository.

Do not restructure the entire project just for the sake of matching this tree.

Preserve useful existing work where possible.

---

38. Important Rule for Implementation

Do not implement Part 2 while working on Part 1.

If you discover something that belongs to Part 2, record it.

For example:

TODO — Part 2

- MQTT TLS

- MQTT ACL

- Prometheus improvements

- OpenTelemetry

- Alerting

- Container security scanning

- Cloud deployment

But do not prematurely introduce unnecessary infrastructure.

---

39. Part 2 Preview

Part 2 will focus on:

Reliability + DevOps

Expected areas:

Secure MQTT

↓

Edge reliability

↓

Observability

↓

Metrics + logs + traces

↓

Security scanning

↓

Container hardening

↓

CI/CD

↓

Cloud deployment

↓

Failure testing

Part 2 will make the system much closer to a **deployable production-oriented platform**.

---

40. Part 3 Preview

Part 3 will focus on:

Deployment + Production Evidence

Potential areas:

Infrastructure as Code

↓

Cloud infrastructure

↓

Container deployment

↓

Optional Kubernetes

↓

Load testing

↓

Backup / restore

↓

Disaster recovery

↓

Chaos testing

↓

SLOs

↓

Production readiness review

Kubernetes, Terraform, GitOps and other tools will only be introduced if they provide meaningful engineering value.

---

41. Final Project Goal

The final goal is **not**:

"I used 20 technologies."

The final goal is:

"I designed, built, secured, tested, observed, deployed and evaluated a distributed edge-cloud system, and I can explain every important engineering decision I made."

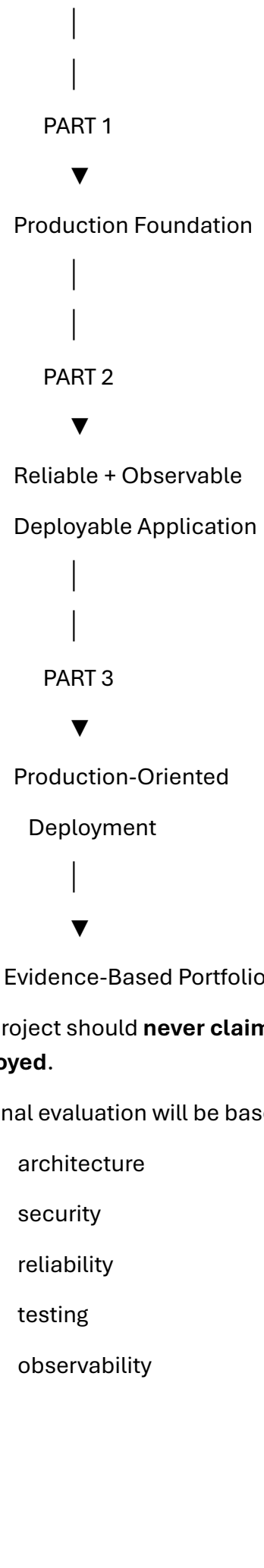
That is the standard EdgeSentinel should aim for.

Current Project Maturity

At the beginning of Part 1:

EdgeSentinel

Functional Prototype



The project should **never claim to be production-ready simply because it is deployed**.

The final evaluation will be based on evidence:

- architecture
- security
- reliability
- testing
- observability

- deployment
- failure recovery
- performance
- documentation
- engineering decisions

That evidence is what makes EdgeSentinel a strong internship project.